Abstract

NimbusCode is a frontend-only, browser-based integrated development environment that enables students to write and execute code without configuring local toolchains. Traditional programming labs often face friction due to compiler installation, operating-system compatibility, and runtime setup differences across devices. NimbusCode addresses this problem by executing supported programming languages inside the browser with WebAssembly-backed runtimes.

The system offers an IDE-inspired user interface containing a file explorer, tabbed Monaco editor, and terminal output panel. It supports JavaScript, Python, C, C++, PHP, SQLite, and Ruby. For C and C++, source code is compiled and linked to WebAssembly within the browser using a runtime pipeline based on clang and wasm-ld binaries. User workspace data is stored locally in IndexedDB, allowing persistence without any backend server.

The project demonstrates a practical local-first educational coding platform that can be deployed as static files. This report analyzes requirements, architecture, implementation, methodology, testing outcomes, limitations, and future scope.

Contents

1 1. INTRODUCTION **1**

1.1 1.1 INTRODUCTION TO THE PROJECT 1

1.2 1.2 STATEMENT OF THE PROBLEM 1

1.3 1.3 SYSTEM SPECIFICATIONS 2

2 2. LITERATURE SURVEY **3**

3 3. SYSTEM ANALYSIS **5**

3.1 3.1 EXISTING SYSTEM 5

3.2 3.2 LIMITATIONS OF THE EXISTING SYSTEM 5

3.3 3.3 PROPOSED SYSTEM 5

3.4 3.4 ADVANTAGES OF THE PROPOSED SYSTEM 6

4 4. SYSTEM DESIGN **8**

4.1 4.1 HIGH LEVEL DESIGN (ARCHITECTURAL) 8

4.2 4.2 LOW LEVEL DESIGN 9

5 7. METHODOLOGY **11**

5.1 7.1 REQUIREMENT ELICITATION AND ANALYSIS 11

5.2 7.2 SYSTEM IMPLEMENTATION APPROACH 11

5.3 7.3 VALIDATION AND VERIFICATION PROCESS 12

5.4 7.4 DOCUMENTATION AND REPORTING METHOD 12

6 8. TESTING **13**

7 9. CONCLUSION **15**

8 10. BIBLIOGRAPHY **16**

9 11. APPENDIX – Sample Source Code/Pseudo Code **17**

List of Tables

1 Supported Languages in NimbusCode 2
2 High-Level Comparison of Development Environments 4
3 Limitations in Conventional Educational Coding Workflows 5
4 Risk and Mitigation Summary 7
5 Requirement Matrix 11
6 Representative Functional Test Cases 13
7 Testing Outcome Summary 14
8 Project Glossary 51
9 Extended Manual Validation Matrix 52

List of Figures

1 NimbusCode High-Level Architectural Blocks 8

1 1. INTRODUCTION

Software development education is strongly influenced by tooling accessibility. Beginners frequently spend significant time configuring runtimes and compilers before writing their first executable program. This setup burden is a major barrier in classrooms, workshops, and self-learning environments. NimbusCode was designed to reduce this burden through a browser-native coding workflow.

NimbusCode treats the browser as the full execution platform. Instead of relying on remote servers for code execution, it uses client-side WebAssembly runtimes. This approach simplifies deployment and improves privacy because source code can remain on the user device.

In addition to execution capability, the project emphasizes interface familiarity. Students are already accustomed to the panel-based layout of modern IDEs. NimbusCode adapts this pattern with an explorer, editor tabs, language awareness, and output console, resulting in a low-friction learning experience.

1.1 1.1 INTRODUCTION TO THE PROJECT

NimbusCode is a static web application developed with React and TypeScript. Its architecture is deliberately backend-free. All primary operations, including file management, editing, and language runtime execution, are performed in the browser context.

The project uses the following technical pillars:

- **UI and state management:** React functional components and hooks.
- **Editor system:** Monaco Editor integration for syntax highlighting and editing.
- **Execution engine:** Runno runtime packages for WebAssembly language support.
- **Storage model:** IndexedDB for local workspace persistence.
- **Build and tooling:** Bun, Vite, TypeScript, ESLint.

NimbusCode currently supports seven languages and extension-based runtime mapping. The implementation includes specialized handling for C and C++ compilation, simple completion providers for selected languages, and configurable editor toggles.

1.2 1.2 STATEMENT OF THE PROBLEM

Educational institutions often face several recurring issues in programming labs:

- Heterogeneous operating systems and environment compatibility problems.
- High onboarding time due to package installations and configuration mismatch.
- System restrictions in lab devices that prevent runtime setup.

- Loss of instructional time resolving machine-specific development issues.
- Difficulty in ensuring consistent execution behavior for all students.

Cloud IDEs partially solve these problems but introduce infrastructure cost, account management, and internet dependency concerns. A fully local browser IDE can provide a middle path: low setup overhead with no backend execution infrastructure.

The problem statement for this project is: *Design and implement a browser-only IDE that supports multi-language execution and persistent workspace management without server-side code execution.*

1.3 1.3 SYSTEM SPECIFICATIONS

Minimum software requirements:

- Modern Chromium-, Firefox-, or WebKit-based browser with WebAssembly support.
- JavaScript enabled and compatibility with ES2022 features.
- Cross-origin isolated serving setup for runtime features requiring shared memory.

Development environment requirements:

- Bun runtime for package management and script execution.
- Node-compatible tooling ecosystem for TypeScript and ESLint.
- Vite development server with configured COOP/COEP headers.

Hardware recommendations for smooth usage:

- Dual-core or better CPU.
- Minimum 4 GB RAM; recommended 8 GB for heavy browser workloads.
- Stable internet connection for first-time runtime asset download.

Table 1: Supported Languages in NimbusCode

Language	Extensions	Runtime Mapping
JavaScript	.js, .mjs, .cjs	quickjs
Python	.py	python
C	.c	clang
C++	.cpp, .cc, .cxx	clangpp
PHP	.php, .phtml	php-cgi
SQLite	.sql	sqlite
Ruby	.rb	ruby

2 2. LITERATURE SURVEY

Browser-based development environments have evolved significantly in the last decade. Early systems mostly offered text editing and remote execution, while modern systems support in-browser runtimes. The emergence of WebAssembly introduced portable high-performance execution in browser sandboxes, enabling interpreters and compilers to run client-side.

2.1 Survey Focus and Evaluation Criteria

The literature and tool survey for this project considered:

- Runtime model (server-side vs client-side execution).
- Setup complexity for beginner users.
- Offline capability and data persistence model.
- Security and isolation model.
- Pedagogical usefulness for introductory programming.

2.2 Comparative Discussion

Traditional desktop IDEs provide rich features and extensibility, but they require local installation and environment setup. In controlled lab settings, this often delays coursework.

Cloud IDEs centralize execution and simplify local setup but require backend resources, account onboarding, and network reliability. They may also introduce privacy concerns because source code is executed remotely.

Notebook platforms are effective for data-oriented workflows but less suitable for file-system-centric multi-language compilation use cases.

WebAssembly-native IDE approach aligns with local-first educational usage by keeping execution in-browser while maintaining an IDE-like interaction model.

Table 2: High-Level Comparison of Development Environments

Aspect	**Desktop IDE**	**Cloud IDE**	**NimbusCode Model**
Setup effort	High (installation needed)	Medium (account and project init)	Low (open browser and start)
Execution location	Local machine	Remote server	Browser runtime (local)
Infra cost	Local maintenance	Continuous cloud cost	Static hosting only
Data location	Local filesystem	Provider-managed cloud storage	Browser IndexedDB
Primary constraint	Device setup complexity	Network and account dependency	Browser runtime limits

2.3 Key Takeaways for This Project

The survey indicates that a frontend-only browser IDE is a valid architectural choice for educational labs, quick experiments, and demonstrations. The most important design challenge is balancing simplicity and capability: adding enough IDE behavior to be useful while avoiding backend complexity.

2.4 References to Enabling Concepts

Core enabling concepts include:

- WebAssembly as a portable low-level execution target.
- WASI for standardized runtime interaction in sandboxed contexts.
- IndexedDB as structured client-side persistence.
- Component-based UI state management for interactive editor systems.

3 3. SYSTEM ANALYSIS

3.1 3.1 EXISTING SYSTEM

In many institutes, practical coding workflows depend on either pre-installed local compilers or web tools that perform remote execution. Existing workflows usually involve at least one of the following pain points:

- Frequent reinstall/configuration cycles across machines.
- Inconsistent language versions between student devices.
- Delays when labs are reset or locked down.
- Limited continuity of student workspace between sessions.

For beginner-focused tasks, the overhead of managing environment details can dominate actual programming time. That mismatch motivated the development of a browser-first workflow.

3.2 3.2 LIMITATIONS OF THE EXISTING SYSTEM

The limitations identified in baseline workflows are summarized below.

Table 3: Limitations in Conventional Educational Coding Workflows

Limitation	**Impact**
Toolchain installation burden	New learners spend excessive time before first successful run.
Version inconsistency	Code that runs on one machine may fail on another.
Administrative restrictions	Lab systems may block installations and runtime updates.
Dependency drift	Environment state changes over time, causing unpredictable failures.
Remote execution dependency	Cloud systems require connectivity and backend reliability.

3.3 3.3 PROPOSED SYSTEM

NimbusCode proposes a local-first architecture where the browser itself is the runtime host. The core solution principles are:

- Keep execution client-side through WebAssembly runtimes.
- Persist workspace state in browser database (IndexedDB).
- Provide a familiar IDE-like interaction model.

- Support multiple languages through extension-based runtime mapping.
- Minimize deployment to static hosting requirements.

3.3.1 Functional Scope

The implemented scope includes:

- File and folder create/delete operations in an explorer tree.
- Tabbed editor workflow with Monaco integration.
- Runtime-aware execution from active file.
- Terminal output visualization and clear action.
- Settings tab for editor preferences (current and future extensibility).
- Local workspace persistence across browser reloads.

3.3.2 Non-Functional Scope

- Frontend-only architecture with no backend service dependency.
- Reasonable responsiveness on commodity hardware.
- Low initial onboarding complexity.
- Reproducible build and lint workflow.

3.4 3.4 ADVANTAGES OF THE PROPOSED SYSTEM

NimbusCode offers clear educational and engineering benefits:

- **Zero local compiler setup** for learners.
- **Consistent workflow** across operating systems with modern browsers.
- **Local privacy model** because source code need not leave the browser.
- **Low deployment complexity** through static hosting.
- **Extensible language architecture** through runtime mapping tables.
- **Immediate usability** for short labs and workshops.

3.4.1 Risk Register and Mitigation

Table 4: Risk and Mitigation Summary

Risk	**Potential Effect**	**Mitigation**
Large bundle size	Higher initial load time	Apply code splitting and lazy runtime loading
Long-running code loops	Browser tab freeze	Add execution interrupt and timeout controls
Browser feature mismatch	Runtime not available on some devices	Validate capabilities and display fallback guidance
Single large component complexity	Maintenance cost increases	Modularize into feature components and hooks

4 4. SYSTEM DESIGN

Design Overview

The system is designed as a single-page application with explicit separation between presentation concerns, workspace persistence, and runtime orchestration.

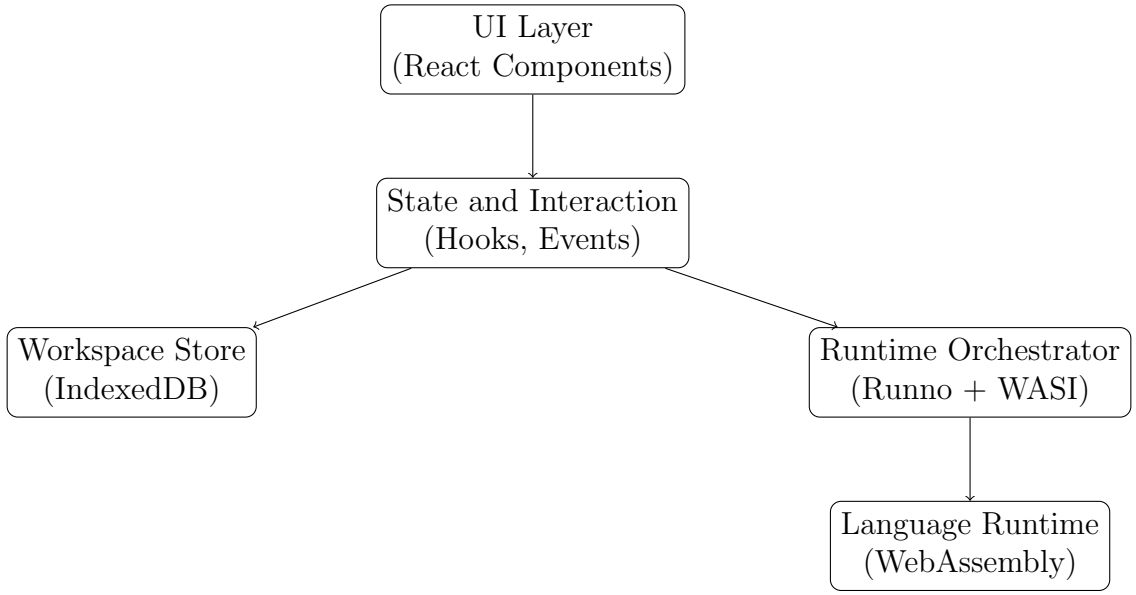

Figure 1: NimbusCode High-Level Architectural Blocks

4.1 4.1 HIGH LEVEL DESIGN (ARCHITECTURAL)

Primary modules in the architecture:

- Explorer and editor interaction layer.
- Runtime selection and execution manager.
- File persistence adapter over IndexedDB.
- Output terminal bridge for run feedback.
- Settings and language metadata controls.

Data flow summary:

1. User selects file in explorer.
2. Active file opens in Monaco editor tab.
3. On run request, extension maps to runtime.
4. Runtime executes code and writes output to terminal.
5. File edits are debounced and persisted in IndexedDB.

4.1.1 Runtime Path for Interpreted Languages

For JavaScript, Python, PHP, Ruby, and SQLite, source text is sent to the relevant runtime through interactive execution API calls. Output streams are rendered in terminal UI. SQLite templates receive placeholder replacement before execution.

4.1.2 Runtime Path for Compiled Languages

For C and C++, NimbusCode runs an in-browser compile pipeline:

1. Build temporary in-memory filesystem with source file.
2. Execute clang in WebAssembly to produce object file.
3. Execute wasm-ld in WebAssembly to produce program.wasm.
4. Run resulting binary in WASI terminal context.

4.2 4.2 LOW LEVEL DESIGN

Low-level design details include path utilities, type models, workspace mutation logic, and UI handlers.

4.2.1 Type Models

Listing 1: Type Models for Workspace Entries

```
export type WorkspaceFileEntry = {
  path: string
  kind: "file"
  content: string
  updatedAt: number
}

export type WorkspaceFolderEntry = {
  path: string
  kind: "folder"
  updatedAt: number
}

export type WorkspaceEntry = WorkspaceFileEntry | WorkspaceFolderEntry
```

4.2.2 Storage Access Layer

Listing 2: IndexedDB Access Pattern

```
const DB_NAME = "nimbuscode-workspace"
const STORE_NAME = "entries"

const openWorkspaceDB = (): Promise<IDBDatabase> =>
  new Promise((resolve, reject) => {
    const request = indexedDB.open(DB_NAME, 1)
    request.onupgradeneeded = () => {
```

```
      const db = request.result
      if (!db.objectStoreNames.contains(STORE_NAME)) {
        db.createObjectStore(STORE_NAME, { keyPath: "path" })
      }
    }
    request.onsuccess = () => resolve(request.result)
    request.onerror = () => reject(request.error)
  })
```

4.2.3 Runtime Mapping Logic

Listing 3: Extension to Runtime Mapping

```
const runtimeByExtension = {
  ".js": "quickjs",
  ".py": "python",
  ".c": "clang",
  ".cpp": "clangpp",
  ".php": "php-cgi",
  ".sql": "sqlite",
  ".rb": "ruby",
}
```

4.2.4 UI State Responsibilities

State variables coordinate selected file path, open tabs, expanded folders, run status, workspace readiness, error messages, and editor completion toggles. This central state approach improves feature velocity but increases component complexity, motivating future modularization.

5 7. METHODOLOGY

5.1 7.1 REQUIREMENT ELICITATION AND ANALYSIS

Methodology began with practical pain-point analysis from educational coding contexts. Functional requirements were captured around four core needs: immediate usability, multi-language support, persistent workspace, and visual workflow familiarity.

Non-functional requirements focused on frontend-only architecture, reproducible builds, and predictable runtime behavior.

7.1.1 Requirement Matrix

Table 5: Requirement Matrix

ID	Requirement	Implementation Evidence
FR-1	Create/delete files and folders	Explorer actions and IndexedDB operations
FR-2	Edit code with syntax support	Monaco editor configuration and language mapping
FR-3	Run active file by runtime mapping	Run button workflow and runtime tables
FR-4	Persist workspace between reloads	IndexedDB store with path-keyed entries
NFR-1	No backend for execution	Browser-only runtime usage
NFR-2	Static deployability	Vite output and static hosting compatibility

5.2 7.2 SYSTEM IMPLEMENTATION APPROACH

Implementation followed an incremental strategy:

1. Establish base project scaffold and lint/type safety.
2. Build explorer and tabbed editor interaction.
3. Integrate persistence APIs for workspace continuity.
4. Integrate runtime execution pipeline per language class.
5. Add UX refinements, settings tab, language metadata, and completion providers.

7.2.1 Iterative Deliverable Model

Each iteration targeted a testable output. This reduced integration risk and made debugging easier. By validating each layer independently (storage, editor, runtime), the project avoided high-cost late-stage rewrites.

5.3 7.3 VALIDATION AND VERIFICATION PROCESS

Verification combined static and dynamic checks:

- TypeScript strict-mode validation through build workflow.
- ESLint checks for code quality and hook correctness.
- Functional testing of file lifecycle and runtime execution.
- Manual regression across language mappings.
- UI interaction validation for tabs, explorer tree, and console output.

7.3.1 Build/Lint Verification Snapshot

Listing 4: Command Verification Snapshot

```
$ bun run lint
$ bun run build
vite v7.x building for production...
built in a few seconds
warning: chunk size exceeds recommended threshold
```

5.4 7.4 DOCUMENTATION AND REPORTING METHOD

Documentation was maintained in markdown and translated to structured report format. Technical notes were grouped into architecture, implementation, testing, and future scope sections. This report consolidates those materials into college-project documentation format.

6 8. TESTING

Testing focused on functional correctness, persistence behavior, runtime stability, and user-flow consistency.

8.1 Test Strategy

- **Unit-level reasoning tests:** Path normalization, runtime mapping, and state transitions.
- **Integration-level tests:** Explorer-to-editor-to-runner flow.
- **Persistence tests:** Browser reload and workspace restoration.
- **Runtime tests:** Language-specific code execution and output checks.

8.2 Representative Test Cases

Table 6: Representative Functional Test Cases

TC	Scenario	Steps	Expected Result
TC1	Create Python file	Create file `main.py`, type code, open tab	File appears in explorer and editor tab
TC2	Run Python code	Click run on active `.py` file	Output appears in terminal
TC3	C compile pipeline	Create `main.c`, run code	Compile then execute within terminal
TC4	Delete folder recursively	Create nested folder and files, delete parent	Parent and child entries removed
TC5	Persist workspace	Reload browser tab	Files/folders restored from IndexedDB
TC6	Unsupported extension handling	Open unsupported file and run	User-visible runtime mapping error
TC7	Clear console action	Produce output, click clear	Terminal resets content panel
TC8	Completion toggle	Disable completions in settings	Suggestions/triggered completion disabled
TC9	Tab close behavior	Open multiple files and close active tab	Next fallback tab becomes active

TC	Scenario	Steps	Expected Result
TC10	SQL placeholder substitution	Run SQL with {{name}} token	Token replaced with safe literal and query runs

8.3 Testing Results Summary

Table 7: Testing Outcome Summary

Area	Status	Notes
Workspace file operations	Passed	Create/edit/delete flows stable in manual tests
Persistence through reload	Passed	IndexedDB entries restored successfully
Runtime execution mapping	Passed	All mapped languages executed with expected routing
Compiled language pipeline	Passed	C/C++ compile and execute path completed
Build and lint checks	Passed	Strict TypeScript and ESLint checks succeeded
Performance optimization	Partial	Large bundle warning observed; needs code splitting

8.4 Identified Defects and Improvements

- Settings text currently says controls are placeholders while completion toggle affects behavior.
- Keybinding mode selector UI exists but has no active runtime behavior mapping.
- Initial production bundle size is larger than recommended threshold.
- No automated test suite integrated yet; validation is currently mostly manual.

8.5 Acceptance Criteria Review

The project meets core acceptance criteria for frontend-only operation, language execution support, persistence, and workflow usability. Additional criteria for advanced editor behavior and testing automation are captured as future improvements.

7 9. CONCLUSION

NimbusCode demonstrates that a practical educational coding environment can be delivered entirely as a frontend application. By combining a familiar IDE workflow with in-browser runtime execution, the project reduces setup friction and enables immediate learning-focused productivity.

The technical implementation validates multiple engineering goals:

- Multi-language execution without backend infrastructure.
- Local persistence through IndexedDB.
- Structured runtime flow for interpreted and compiled languages.
- Static deployment readiness with reproducible build pipeline.

From a pedagogy perspective, the system is suitable for introductory labs, workshops, and demonstrations where rapid onboarding is critical. From a software engineering perspective, the project can evolve into a larger platform through component modularization, automated tests, and advanced editor integrations.

Future Scope

- Implement actual Vim/Emacs keybinding integration.
- Add execution interruption control for long-running programs.
- Introduce lazy loading and manual chunk optimization.
- Add import/export workspace snapshots.
- Add optional sync mode while preserving local-first default.
- Introduce unit/integration test suites in CI.

8 10. BIBLIOGRAPHY

1. React Documentation. https://react.dev/
2. TypeScript Handbook. https://www.typescriptlang.org/docs/
3. Vite Documentation. https://vite.dev/guide/
4. Bun Documentation. https://bun.sh/docs
5. Monaco Editor. https://github.com/microsoft/monaco-editor
6. Runno Runtime. https://github.com/taybenlor/runno
7. WebAssembly Official Concepts. https://webassembly.org/docs/
8. MDN Web Docs: IndexedDB API. https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
9. W3C Cross-Origin Isolation guidance and browser compatibility notes.
10. Course notes and institutional lab guidelines for web systems projects.

9 11. APPENDIX – Sample Source Code/Pseudo Code

Appendix A: Pseudo Code for Workspace Loading

Listing 5: Pseudo Code – Workspace Initialization

```
function loadWorkspace():
    persisted = listWorkspaceEntries()
    if persisted is empty:
        source = initialWorkspace
        putWorkspaceEntries(source)
    else:
        source = persisted

    firstFile = first entry in source where kind == "file"
    setEntries(sort(source))
    setSelectedPath(firstFile.path)
    setActiveFilePath(firstFile.path)
    setOpenTabs([firstFile.path])
```

Appendix B: Pseudo Code for C/C++ Runtime Pipeline

Listing 6: Pseudo Code – Compiled Language Execution

```
function runCompiled(runtime, code):
    fs = { sourceFilePath(runtime): code }

    for step in prepareCommands(runtime):
        if step requires base filesystem:
            fs += fetchBaseFilesystem(step.baseFSURL)

        result = WASI.start(step.binary, args=step.args, fs=fs)
        if result.exitCode != 0:
            return failure("Compile step failed")
        fs = result.fs

    binary = readBinary(fs, "/program.wasm")
    if binary not found:
        return failure("No wasm produced")

    runResult = terminal.run(binary)
    return runResult
```

Appendix C: Core Storage Module Excerpt

Listing 7: IndexedDB Operations Excerpt

```
export const putWorkspaceEntries = async (entries) => {
  if (entries.length === 0) return
  const db = await openWorkspaceDB()
  try {
    const transaction = db.transaction("entries", "readwrite")
```

```
    const store = transaction.objectStore("entries")
    for (const entry of entries) {
      store.put(entry)
    }
    await transactionDone(transaction)
  } finally {
    db.close()
  }
}

export const deleteWorkspacePaths = async (paths) => {
  if (paths.length === 0) return
  const db = await openWorkspaceDB()
  try {
    const transaction = db.transaction("entries", "readwrite")
    const store = transaction.objectStore("entries")
    for (const path of paths) {
      store.delete(path)
    }
    await transactionDone(transaction)
  } finally {
    db.close()
  }
}
```

Appendix D: Engineering Notes and Traceability

The following appendix pages document iteration-level progress logs, engineering observations, and deliverable checkpoints. These pages provide traceability for project execution across planning, implementation, validation, and documentation phases.

Appendix E.1: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 1 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 1. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 1 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.2: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 2 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 2. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 2 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.3: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 3 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 3. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 3 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.4: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 4 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 4. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 4 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.5: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 5 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 5. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 5 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.6: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 6 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 6. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 6 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.7: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 7 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 7. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 7 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.8: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 8 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 8. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 8 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.9: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 9 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 9. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 9 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.10: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 10 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 10. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 10 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.11: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 11 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 11. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 11 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.12: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 12 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 12. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 12 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.13: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 13 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 13. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 13 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.14: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 14 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 14. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 14 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.15: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 15 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 15. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 15 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.16: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 16 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 16. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 16 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.17: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 17 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 17. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 17 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.18: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 18 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 18. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 18 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.19: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 19 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 19. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 19 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.20: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 20 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 20. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 20 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.21: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 21 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 21. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 21 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.22: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 22 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 22. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 22 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.23: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 23 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 23. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 23 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.24: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 24 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 24. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 24 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.25: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 25 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 25. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 25 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.26: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 26 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 26. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 26 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.27: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 27 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 27. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 27 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.28: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 28 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 28. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 28 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.29: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 29 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 29. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 29 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.30: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 30 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 30. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 30 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.31: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 31 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 31. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 31 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix E.32: Iteration and Progress Log

Iteration Goal: Stabilize NimbusCode iteration 32 for academic reporting

Primary Tasks Completed: Reviewed explorer interactions, editor state behavior, runtime execution flow, and persistence checks for iteration 32. Updated implementation notes and synchronized source observations with the project report.

Observations and Outcomes: Iteration 32 improved documentation completeness, clarified known limitations, and preserved reproducible verification steps for build, lint, and runtime validation.

Deliverables recorded for this iteration:

- Updated source modules and refactored client-side workflow.
- Regression checks through manual run/lint/build verification.
- Documentation updates reflecting architectural and testing changes.
- Risk log updated with mitigation notes and pending improvements.

Status: Completed and documented for college project submission.

Appendix F: Glossary of Terms

Table 8: Project Glossary

Term	Meaning in This Project
WebAssembly (Wasm)	Portable binary instruction format used for in-browser execution.
WASI	WebAssembly System Interface for sandboxed system-level interactions.
Runtime Mapping	Association of file extension with the execution runtime.
IndexedDB	Browser-native structured storage used for workspace persistence.
Workspace Entry	Data entity representing either a file or a folder path.
Explorer Tree	Left panel that displays folder and file hierarchy.
Terminal Panel	Output area where runtime stdout/stderr is displayed.
Completion Provider	Monaco integration that provides language suggestions/snippets.
Compile Pipeline	Multi-step process for C/C++ code to produce executable Wasm.
Cross-Origin Isolation	Header-based browser security mode enabling shared memory features.
COOP	Cross-Origin-Opener-Policy header used for isolation.
COEP	Cross-Origin-Embedder-Policy header used for isolation.
Static Hosting	Deployment model where built frontend assets are served without backend logic.
Debounced Save	Delayed persistence write to reduce frequent storage transactions.
Tab Activation Flow	Logic for selecting fallback tabs when active tab is closed.

Appendix G: Additional Test Artifact Matrix

Table 9: Extended Manual Validation Matrix

ID	Validation Item	Result	Remarks
M1	Open language dropdown from navbar	Pass	Supported list visible and dismisses on blur
M2	Select folder in explorer tree	Pass	Ancestor expansion state maintained
M3	Create nested folder path using inline input	Pass	Parent folders auto-created where needed
M4	Create file from selected folder context	Pass	Path joined relative to selected directory
M5	Update file content and wait save debounce	Pass	Entry persisted after timeout
M6	Delete selected file with open tab	Pass	Tab removed and fallback activated
M7	Delete selected folder recursively	Pass	Child entries deleted from store
M8	Run JavaScript snippet	Pass	Output printed in terminal
M9	Run Python snippet	Pass	Prompt/input workflow available
M10	Run PHP program through php-cgi runtime	Pass	Program output displayed
M11	Run Ruby script	Pass	Runtime output displayed
M12	Run SQLite query with placeholder token	Pass	Token replacement path executed
M13	Run C code via compile/link pipeline	Pass	Binary generated and executed
M14	Run C++ code via compile/link pipeline	Pass	Binary generated and executed
M15	Trigger runtime mapping error on unknown extension	Pass	Error message shown in terminal
M16	Toggle completion switch in settings	Pass	Suggest settings updated in editor options

ID	Validation Item	Result	Remarks
M17	Change keybinding select option	Pass	UI state changes (behavior mapping pending)
M18	Clear terminal output and rerun	Pass	Fresh output panel observed
M19	Reload page with existing workspace	Pass	Files restored from IndexedDB
M20	Production build check	Pass	Build completes with large chunk warning

Appendix H: Extended Discussion on Maintainability

NimbusCode currently consolidates most UI and orchestration logic in a single large component. This helps rapid prototyping but increases cognitive load for future contributors. A maintainability-focused refactor should split responsibilities into dedicated modules such as ExplorerController, RuntimeRunner, EditorTabs, WorkspaceService, and SettingsPanel.

A modular architecture would improve testability and reduce accidental coupling between view-state transitions and runtime events. Hooks can encapsulate behaviors like tab lifecycle, persistence debounce, and completion provider registration. This report recommends refactoring in phases to avoid behavior regressions.

Appendix I: Deployment Checklist

1. Install project dependencies with `bun install`.
2. Verify lint and type checks using `bun run lint` and `bun run build`.
3. Ensure static host serves COOP/COEP headers where required.
4. Deploy generated `dist/` files to static hosting platform.
5. Validate runtime behavior in target browser set.
6. Confirm IndexedDB persistence and permission behavior.
7. Capture post-deployment smoke test evidence.

Appendix J: Sample Command Log

Listing 8: Development Command Log Example

```
$ bun install
$ bun run dev
$ bun run lint
$ bun run build
$ bun run preview
```

Appendix K: Final Remarks for Submission

This report is prepared in the requested structure and style for college project submission. It includes architectural reasoning, implementation evidence, testing artifacts, bibliography, and appendices that support both technical review and academic evaluation.